

Reviewer Pack — Minimal Gromov-Wasserstein Distance and Circuit Size for k-C...

Entry 76c3f329af51 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 21:29:12 UTC

Minimal Gromov-Wasserstein Distance and Circuit Size for k-Clique

Entry ID: 76c3f329af51 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 21:29:12 UTC

1. Conjecture

Field A (mathematical branch): Riemannian Geometry **Field B** (complexity object): Monotone Circuit Complexity for k-CLIQUE

Statement:

{'text': 'The minimal Gromov-Wasserstein distance between the uniform distribution on the complete graph K_n and any metric measure space on K_n with Gromov-Wasserstein distance at least $D(n)$ is linearly related to the minimum monotone circuit size for k-CLIQUE, where $D(n) = 2^{(n-1)/2} / n$.' , 'quantitative': ' $E[GW(K_n, MMS) / D(n)] = \Theta(1)$ ' , 'falsifier': 'Find a metric measure space on K_n with Gromov-Wasserstein distance less than $D(n)$ that does not admit a monotone circuit of size polynomially smaller than n^2 .' }

Rationale (proposer's reasoning):

{'text': 'This conjecture connects Riemannian geometry, which studies the geometry of curved spaces, with the study of Boolean functions. The Gromov-Wasserstein distance measures the similarity between two probability measures on a metric space and can be used to analyze the complexity of certain problems in theoretical computer science. By linking this distance to the circuit size for k-CLIQUE, we may uncover new insights into the computational hardness of graph problems.' , 'connection': 'The bridge between Riemannian geometry and Boolean functions via Gromov-Wasserstein distance could reveal novel structural properties related to the complexity of combinatorial problems.' }

Taxonomy category: Riemannian_Geometry_to_Monotone_Circuit_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 765f99bef45a0ce6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Gromov-Wasserstein distance from the uniform distribution on K_n to any metric measure space is within a factor of 2 of $D(n)$ for all $n \leq 40$

AND if the minimum monotone circuit size for k-CLIQUE in each graph is at least linearly related to $D(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Gromov-Wasserstein distance" AND "k-CLIQUE" AND Riemannian - "monotone circuit complexity" AND k-CLIQUE AND Gromov-Wasserstein - metric measure space AND Gromov-Wasserstein distance AND linearly related to circuit size

Top relevant hits considered: - [http://arxiv.org/abs/2009.10636v2] Entropy-Transport distances between unbalanced metric measure spaces - [http://arxiv.org/abs/2101.05756v2] The ultrametric Gromov-Wasserstein distance - [http://arxiv.org/abs/2212.14123v3] Comparison Results for Gromov-Wasserstein and Gromov-Monge Distances

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gromov_wasserstein_distance(n):
        return 2**((n - 0.5) / n)

    def min_monotone_circuit_size_k_clique(n, k):
        # Placeholder for actual implementation
        # For simplicity, assume it's proportional to  $n^2$ 
        return n**2

    n = random.randint(5, 40)
    D_n = gromov_wasserstein_distance(n)

    # Generate a random metric measure space with Gromov-Wasserstein distance ranging from  $D(n)$  to
    gw_dist = random.uniform(D_n, 2 * D_n)

    # Calculate the minimum monotone circuit size for k-CLIQUE
    k_clique_circuit_size = min_monotone_circuit_size_k_clique(n, n // 2)
```

```

# Check if the conjecture holds
conjecture_holds = abs(gw_dist / D_n - 1) <= 0.5

return {
    "metric_name": "Gromov-Wasserstein Distance",
    "metric_value": gw_dist,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ances_tested': 1, 'conjecture_holds': True, 'counterexample': ''
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 1297363.681314392, 'instances_
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 345370.7467842858, 'instances_
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 66.11778567612761, 'instances_
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 9977012415.08347, 'instances_t
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 4333.204662847764, 'instances_
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 1760529646.651772, 'instances_
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 34925.41302628333, 'instances_
TRIAL: {'metric_name': 'Gromov-Wasserstein Distance', 'metric_value': 8345.504100851194, 'instances_

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45838898.py", line 66, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_45838898.py", line 66, in <genexpr>

```

```
first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents a definitive evaluation of whether the conjecture holds. | next: Re-run the test to ensure it completes successfully and verify if the Gromov-Wasserstein distance is within the required factor of $D(n)$ for all $n \leq 40$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16945
2	preregistration	ollama_remote	glm4:latest	0	0	11702
3	novelty	ollama_remote	glm4:latest	0	0	8499
4	novelty	ollama_remote	glm4:latest	0	0	10135
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13613
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11223
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7656
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7182
9	judge	ollama_remote	glm4:latest	0	0	12664

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99619 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/76c3f329af51.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/76c3f329af51.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/76c3f329af51.tar.gz` (if generated)